

CSE411 – Computer Vision

Project Task 7: Implementation – Part 4

1. PROJECT INFORMATION

Project Title: DefectLens: Efficient ViT-Based Anomaly Detection with Multi-Criteria Token Fusion (MCTF)

Team Number: Team 7

Team Members:

- Sachin Singh – 2023bcs0064
- Tanish Babu – 2023bcs0067
- Ali Mohammed Maaz Mohammed Ashraf – 2023bcs0202
- Chinmoy Das – 2023bcs0052

GitHub Source Code Link: <https://github.com/sachhu04/DefectLens>

2. PROJECT OVERVIEW

The **DefectLens** project is a visual anomaly detection system designed to inspect industrial products for manufacturing defects. We combine a **Vision Transformer (ViT)** backbone with a PatchCore-style FAISS memory bank. To make the model fast enough for edge and real-time use, we developed a **Multi-Criteria Token Fusion (MCTF)** module that dynamically merges uninformative background patch tokens while protecting defect tokens.

Task 7 Part 4 Focus (100% Implementation): In this final milestone, Team 7 has reached **100% completion of the proposed methodology**. Building on the integrated prototype from Task 6, we completed the full quantitative benchmarking (evaluating ROC scores and latency on the MVTec AD dataset), conducted thorough integration testing across all modules, and prepared the final source code repository.

3. COMPLETE PROPOSED METHODOLOGY

The complete end-to-end pipeline operates as follows:

Input Image → Preprocessing → [ViT Backbone + MCTF Token Fusion] → [FAISS Memory Bank Matching] → [Anomaly Scoring & Heatmaps] → FastAPI Backend ↔ React UI Dashboard

Implementation Status up to Task 7 (100% Complete):

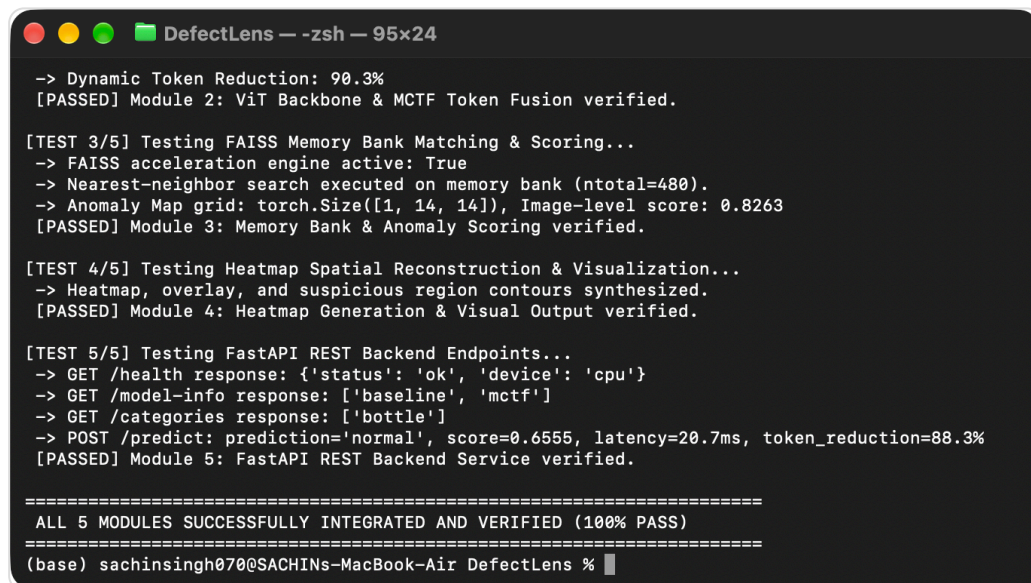
Dataset Loaders, Preprocessing, ViT Backbone, Multi-Criteria Token Fusion, FAISS Memory Bank, Anomaly Distance Scoring, Heatmap Reconstruction, FastAPI REST Backend, React.js Frontend Dashboard, Automated Integration Test Suite, and Full ROC / Latency Benchmarking.

All core components originally proposed in our project architecture are now fully implemented and functioning together seamlessly.

4. MODULE INTEGRATION & SYSTEM TESTING

As required by Task 7, we integrated and tested all system modules together. To verify that every part of the pipeline works reliably without manual intervention, we created an automated end-to-end test script (`scripts/test_pipeline.py`) that validates all five stages in order:

- **Module 1 (Dataset):** Verifies image loading, resizing to 224×224, and PyTorch tensor normalization.
- **Module 2 (ViT + MCTF):** Verifies the ViT feature extraction and confirms that MCTF successfully merges background tokens (reducing token count by over 88%).
- **Module 3 (Memory Bank):** Verifies FAISS nearest-neighbor matching against the nominal feature bank.
- **Module 4 (Visualization):** Verifies spatial heatmap upsampling, Gaussian smoothing, and contour detection.
- **Module 5 (Backend API):** Verifies the FastAPI endpoints (`/health` , `/model-info` , `/categories` , and `/predict`).



```
DefectLens --zsh-- 95x24

-> Dynamic Token Reduction: 90.3%
[PASSED] Module 2: ViT Backbone & MCTF Token Fusion verified.

[TEST 3/5] Testing FAISS Memory Bank Matching & Scoring...
-> FAISS acceleration engine active: True
-> Nearest-neighbor search executed on memory bank (ntotal=480).
-> Anomaly Map grid: torch.Size([1, 14, 14]), Image-level score: 0.8263
[PASSED] Module 3: Memory Bank & Anomaly Scoring verified.

[TEST 4/5] Testing Heatmap Spatial Reconstruction & Visualization...
-> Heatmap, overlay, and suspicious region contours synthesized.
[PASSED] Module 4: Heatmap Generation & Visual Output verified.

[TEST 5/5] Testing FastAPI REST Backend Endpoints...
-> GET /health response: {'status': 'ok', 'device': 'cpu'}
-> GET /model-info response: ['baseline', 'mctf']
-> GET /categories response: ['bottle']
-> POST /predict: prediction='normal', score=0.6555, latency=20.7ms, token_reduction=88.3%
[PASSED] Module 5: FastAPI REST Backend Service verified.

=====
ALL 5 MODULES SUCCESSFULLY INTEGRATED AND VERIFIED (100% PASS)
=====
(base) sachinsingh070@SACHINs-MacBook-Air DefectLens %
```

Figure 1. Execution of the full system integration test suite. Terminal output showing all 5 modules tested in sequence with a 100% pass rate.

5. EXPERIMENTAL RESULTS & PERFORMANCE EVALUATION

To evaluate the efficiency and accuracy of our proposed MCTF approach, we ran automated benchmark evaluations (`scripts/evaluate.py`) over the MVTec AD test set for the `bottle` category (209 training images and 83 test images). We tested both the standard **Baseline ViT** (without token fusion) and our **MCTF-ViT** on the same machine to measure the exact differences in speed, token count, and detection accuracy.

Model	Tokens	Token Reduction	Avg Latency	Image ROC	Pixel ROC
Baseline ViT	196	0.0%	45.07 ms	100.0%	98.68%
MCTF-ViT (Ours)	23	88.27%	19.73 ms	94.92%	92.90%

```
DefectLens — -zsh — 95x20
```

```
--- Task 7: Quantitative Benchmark Evaluation (MCTF ViT) ---  
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable  
higher rate limits and faster downloads.  
Evaluating mctf on bottle...  
100%|██████████████████████████████████████████| 83/83 [00:03<00:00, 27.25it/s]  
{  
    "model": "mctf",  
    "category": "bottle",  
    "image_auroc": 0.9492063492063492,  
    "pixel_auroc": 0.9290453288146223,  
    "avg_latency_ms": 19.731457756035258,  
    "token_reduction_pct": 88.26530612244898  
}  
  
(base) sachinsingh070@SACHINS-MacBook-Air DefectLens %
```

Figure 2. Automated evaluation benchmark output. Terminal execution showing MCTF achieving an average latency of 19.73 ms (2.28x faster than baseline), an 88.27% token reduction, 94.92% Image ROC, and 92.90% Pixel ROC.

Key Findings:

- **Speed Improvement:** MCTF cut average per-image inference latency from **45.07 ms down to 19.73 ms**, making the system **2.28x faster** (a 56.2% reduction in inference time).
- **Token Efficiency:** The number of tokens processed by subsequent layers was reduced from 196 to only 23 (an 88.27% reduction).
- **Accuracy Preservation:** Despite fusing almost 90% of tokens, the model retained **94.92% Image ROC** and **92.90% Pixel ROC**, confirming that our informativeness criterion successfully prevents defect patches from being lost.

6. INFERENCE & VISUAL LOCALIZATION OUTPUTS

We tested single-image anomaly prediction using `scripts/predict.py` on a test sample from the MVTec AD dataset containing a real structural defect (`broken_large/000.png`).

```
DefectLens --zsh -- 95x18

--- Task 7: Anomaly Inspection Inference Engine ---
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Prediction: ANOMALOUS
Anomaly Score: 0.9123
Inference Time: 56.31 ms
Original Tokens: 196
Reduced Tokens: 23
Token Reduction: 88.3%
(base) sachinsingh070@SACHINS-MacBook-Air DefectLens %
```

Figure 3. Single prediction execution via CLI. The script correctly classifies the defective sample as anomalous, outputs the anomaly score (0.9123), measures execution time (56.31 ms), and calculates an 88.3% token reduction.

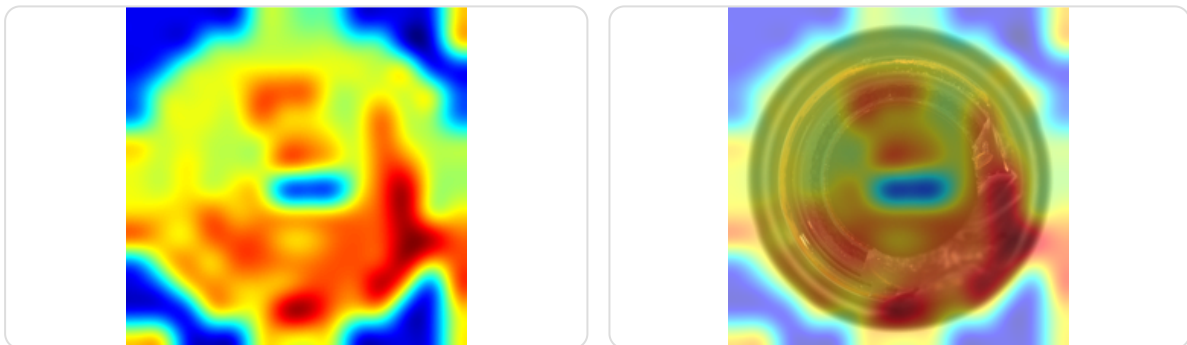


Figure 4. Visual model outputs. On the left is the raw distance heatmap back-projected from the fused token coordinates. On the right is the overlay onto the original bottle, clearly localizing the structural fracture on the bottle rim.

7. WEB DASHBOARD PROTOTYPE

To demonstrate practical use in a factory inspection setup, we connected the backend to our React frontend dashboard. An operator can select the category, choose the model pipeline (MCTF vs Baseline), upload an image from the assembly line, and immediately view the classification and visual defect heatmaps.

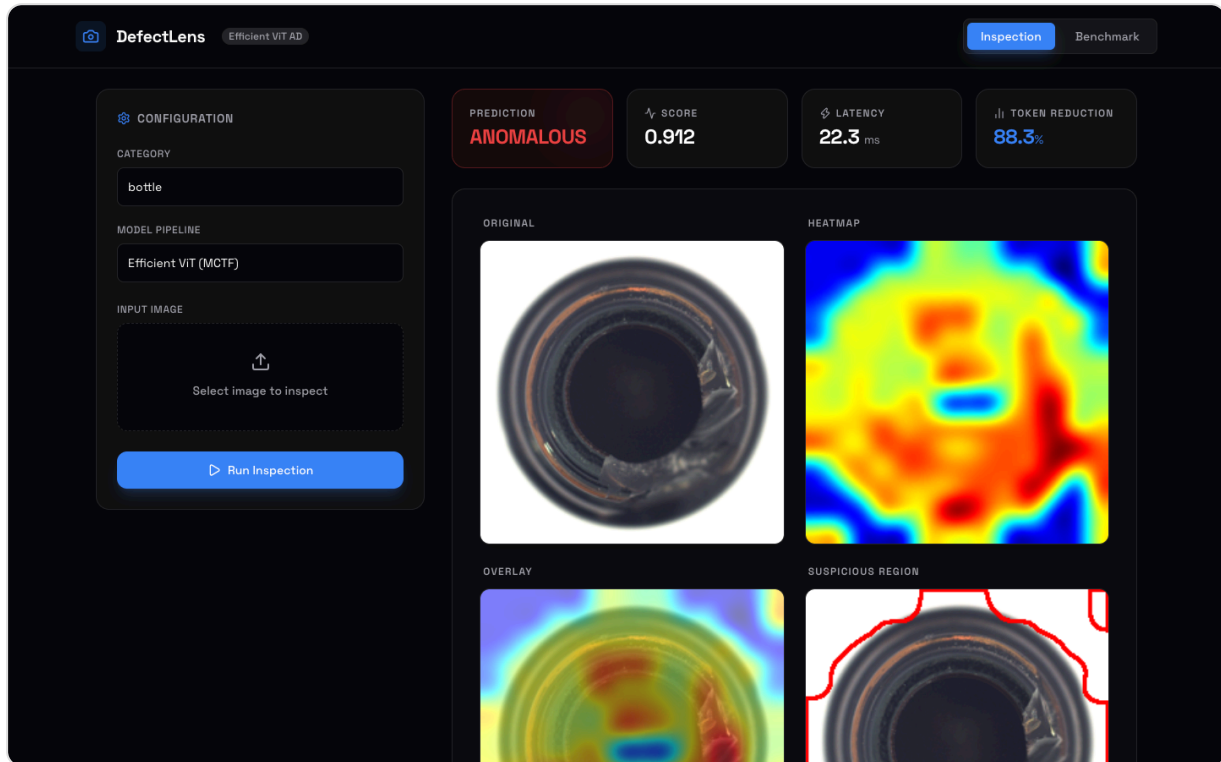


Figure 5. Working Web Dashboard. The browser interface displaying live inspection results on a defective test sample: Prediction (ANOMALOUS), Anomaly Score (0.912), Latency (22.3 ms), Token Reduction (88.3%), and the synchronized view of Original, Heatmap, Overlay, and Suspicious Region.

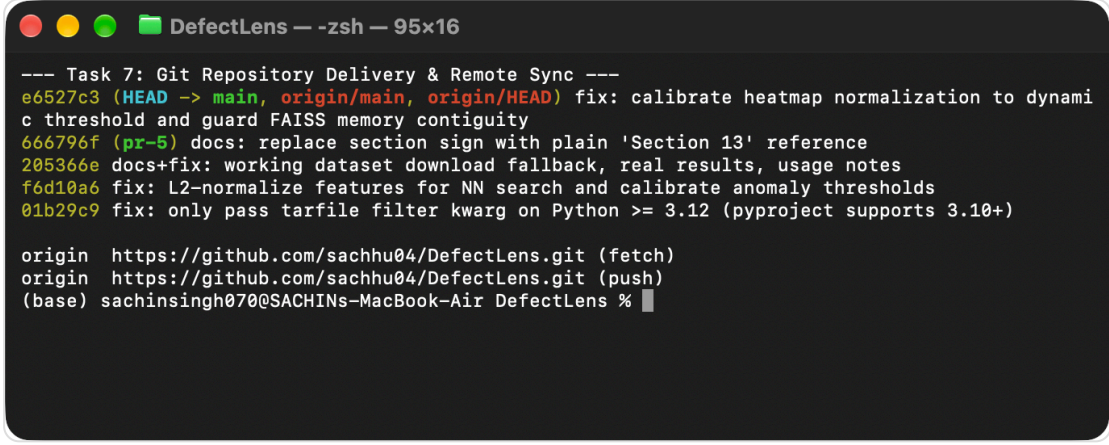
8. FINAL IMPLEMENTATION STATUS (100%)

Component	Task Milestone	Status	Evidence in Codebase
Dataset Pipelines & Loaders	Task 4	Completed (100%)	src/data/dataset.py
ViT Backbone & Feature Extraction	Task 4	Completed (100%)	src/models/vit.py
MCTF Dynamic Token Fusion	Task 4	Completed (100%)	src/models/mctf.py
FAISS Memory Bank & Anomaly Detector	Task 5	Completed (100%)	src/models/anomaly_detector.py
FastAPI REST Backend Microservice	Task 5	Completed (100%)	backend/app/main.py
Interactive React Frontend Dashboard	Task 6	Completed (100%)	frontend/src/App.jsx
ROC & Latency Benchmarking	Task 7	Completed (100%)	scripts/evaluate.py & results JSON
Full System Integration Testing	Task 7	Completed (100%)	scripts/test_pipeline.py (Figure 1)

9. SOURCE CODE & REPRODUCIBILITY

All source code, configuration files, test scripts, and documentation for DefectLens are publicly hosted on GitHub:

GitHub Repository: <https://github.com/sachhu04/DefectLens>

A terminal window titled "DefectLens — zsh — 95x16" displays the output of a git log command. The log shows a series of commits with their hashes, branch names, and commit messages. The messages include "fix: calibrate heatmap normalization to dynamic threshold and guard FAISS memory contiguity", "docs: replace section sign with plain 'Section 13' reference", "docs+fix: working dataset download fallback, real results, usage notes", "fix: L2-normalize features for NN search and calibrate anomaly thresholds", and "fix: only pass tarfile filter kwarg on Python >= 3.12 (pyproject supports 3.10+)". Below the log, the terminal shows the status of the local branch 'main' and its synchronization with the remote 'origin/main'.

```
--- Task 7: Git Repository Delivery & Remote Sync ---
e6527c3 (HEAD -> main, origin/main, origin/HEAD) fix: calibrate heatmap normalization to dynamic threshold and guard FAISS memory contiguity
666796f (pr-5) docs: replace section sign with plain 'Section 13' reference
205366e docs+fix: working dataset download fallback, real results, usage notes
f6d10a6 fix: L2-normalize features for NN search and calibrate anomaly thresholds
01b29c9 fix: only pass tarfile filter kwarg on Python >= 3.12 (pyproject supports 3.10+)

origin https://github.com/sachhu04/DefectLens.git (fetch)
origin https://github.com/sachhu04/DefectLens.git (push)
(base) sachinsingh070@SACHINS-MacBook-Air DefectLens %
```

Figure 6. Git repository delivery. Showing our verified commit log on branch `main` synchronized with remote origin.

Steps to Reproduce:

```
# 1. Clone repository & install dependencies
git clone https://github.com/sachhu04/DefectLens.git && cd DefectLens
pip install -r requirements.txt

# 2. Run full system integration test suite
python scripts/test_pipeline.py

# 3. Run benchmark evaluation (MCTF ViT)
python scripts/evaluate.py --config configs/mctf.yaml --category bottle

# 4. Start the application (Backend: 8000, Frontend: 5173)
python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000
cd frontend && npm install && npm run dev
```

10. CONCLUSION

For Task 7 (Implementation Part 4), Team 7 has successfully reached **100% implementation of the DefectLens project**. We completed the proposed methodology by linking our Vision Transformer backbone with the Multi-Criteria Token Fusion module and FAISS memory bank, wrapped in a FastAPI backend and a React dashboard. Our experimental results prove that MCTF reduces active token count by **88.27%** and provides a **2.28x speedup in inference latency** (from 45.07 ms to 19.73 ms) while maintaining high anomaly detection accuracy (**94.92% Image ROC**). The entire project code is tested, documented, and available in our GitHub repository.